

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №7
по дисциплине
«Программирование»
по теме: "СОЗДАНИЕ ИНТЕРАКТИВНОГО ДАШБОРДА"

Выполнил студент гр. ИВТб-1301-06-00	_____ /Габдулбариев Т.Р./
Проверил доцент кафедры ЭВМ	_____ /Долженкова М.Л./

Киров
2026

Цель

Освоение практических навыков создания интерактивных дашбордов в Python, включая: интеграцию pandas DataFrame с графическими библиотеками matplotlib и seaborn; встраивание графиков в окно Tkinter через бэкенд FigureCanvasTkAgg; реализацию событийно-ориентированной архитектуры (callbacks, event loop); динамическую фильтрацию, агрегацию и feature engineering «на лету»; проектирование устойчивого процедурного кода без использования классов.

Repository: <https://github.com/kokoshechka732/laba7>

Описание варианта

Предметная область: аналитика публикаций в социальных сетях для оценки вовлеченности аудитории, вирусного потенциала контента и поведенческих паттернов пользователей. Поля данных: ts - время публикации, acc_id - идентификатор аккаунта, likes - количество лайков, shares - число репостов, com - количество комментариев, sent - индекс тональности контента от -1 до 1. Очистка данных включает установку отрицательных лайков в ноль, ограничение тональности диапазоном $[-1, 1]$ и комментариев по 99-му перцентилю.

Расчетные метрики: engagement как сумма лайков, репостов и комментариев; virality через разницу репостов между публикациями аккаунта; engagement_rate с защитой от деления на ноль; post_quality для классификации постов на вирусные, интересные и обычные по пороговым значениям 75-го перцентиля.

Условия фильтрации: выбор аккаунта из списка или все аккаунты, фильтр по качеству поста, выбор целевой метрики для анализа, выбор типа агрегации данных.

Типы графиков: линейный для отображения динамики метрики по дням с возможностью агрегации, столбчатый для показа топ-10 аккаунтов по

выбранному показателю, точечный для визуализации зависимости лайков от ID поста с цветовой кодировкой по качеству, тепловая карта для анализа корреляций между числовыми признаками.

Исходный код блока кода на Python

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

sns.set_theme(style="whitegrid")
plt.rcParams["font.family"] = "DejaVu Sans"

global cdr

data = np.load("data_final.npy", allow_pickle=True)
df_raw = pd.DataFrame(data)

df_raw["post_id"] = range(1, len(df_raw) + 1)

df_raw["ts"] = pd.date_range(
    start="2026-05-01",
    end="2026-05-31 23:59:59",
    periods=len(df_raw)
)
df_raw["ts"] = pd.to_datetime(df_raw["ts"])

df_raw["acc_id"] = df_raw["acc_id"].astype(str)
```

Описание блока кода

Задаются глобальные стили для графиков. Далее из бинарного .npy файла загружается сырой массив данных, который преобразуется в DataFrame. Для каждого поста генерируется уникальный идентификатор (post_id), создается временной ряд, распределенный равномерно на май 2026 года, а идентификаторы аккаунтов приводятся к строковому типу для корректной работы будущих фильтров.

Исходный код блока кода на Python

```
def updatec(dataf):
    dataf = dataf.copy()

    dataf["engagement"] = (
        dataf.get("likes", 0) + dataf.get("shares", 0) + dataf.get("com", 0)
    )

    dataf = dataf.sort_values(["acc_id", "ts"])

    dataf["virality"] = (
        dataf.groupby("acc_id")["shares"].diff().fillna(0)
    )

    dataf["engagement_rate"] = dataf["engagement"] / (dataf["engagement"].max() + 1)

    high_eng = dataf["engagement"].quantile(0.75)
    high_vir = dataf["virality"].quantile(0.75)

    conditions = [
        (dataf["engagement"] > high_eng) & (dataf["virality"] > high_vir),
        (dataf["engagement"] > high_eng)
    ]
    choices = ["Вирусный", "Интересный"]
```

```

dataf["post_quality"] = np.select(
    conditions, choices, default="Обычный"
)

return dataf

dataf_work = updatec(df_raw)

```

Описание блока кода

Функция `updatec` рассчитывает новые метрики: **Engagement**: сумма лайков, репостов и комментариев. **Virality**: изменение количества репостов внутри каждого аккаунта. **Engagement Rate**: нормализованный коэффициент вовлеченности относительно максимума. **Post Quality**: автоматическая классификация постов на три категории на основе пороговых значений (75-й перцентиль) для вовлеченности и виральности с помощью `np.select`.

Исходный код блока кода на Python

```

root = tk.Tk()
root.title("Дашборд: Анализ активности")
root.geometry("1300x850")

main = tk.Frame(root)
main.pack(fill=tk.BOTH, expand=True)

left = tk.Frame(main, width=250)
left.pack(side=tk.LEFT, fill=tk.Y, padx=10, pady=10)

right = tk.Frame(main)
right.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=10, pady=10)

fig = plt.figure(figsize=(9, 5), dpi=100)
canvas = FigureCanvasTkAgg(fig, master=right)
canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

```

```
toolbar = NavigationToolbar2Tk(canvas, right)
toolbar.update()
toolbar.pack(side=tk.TOP, fill=tk.X)
```

```
acc_var      = tk.StringVar(value="Bce")
metric_var   = tk.StringVar(value="engagement")
agg_var      = tk.StringVar(value="sum")
quality_var  = tk.StringVar(value="Bce")
cur_chart    = "line"
```

Описание блока кода

Создается главное окно, которое разделяется на две основные зоны: левую панель фиксированной ширины и правую область. В правую часть встраивается фигура Matplotlib через FigureCanvasTkAgg, а также добавляется стандартная панель инструментов NavigationToolbar2Tk для масштабирования и сохранения графиков. Инициализируются переменные состояния, которые будут хранить текущие выборы пользователя в выпадающих списках и глобальная переменная `cur_chart` для отслеживания активного типа диаграммы.

Исходный код блока кода на Python

```
def get_filtered():
    dataf = dataf_work.copy()
    acc = acc_var.get()
    if acc != "Bce":
        dataf = dataf[dataf["acc_id"] == acc]
    post_q = quality_var.get()
    if post_q != "Bce":
        dataf = dataf[dataf["post_quality"] == post_q]
    return dataf
```

```

def clear():
    fig.clear()

def show_empty():
    clear()
    ax = fig.add_subplot(111)
    ax.text(0.5, 0.5, "Нет данных", ha="center", va="center", fontsize=14)
    canvas.draw_idle()

def plot_line():
    global cur_chart
    cur_chart = "line"
    dataf = get_filtered()
    if dataf.empty:
        show_empty()
        return
    clear()
    metric = metric_var.get()
    agg = agg_var.get()
    dataf_daily = (
        dataf.set_index("ts").resample("D")[metric].agg(agg).reset_index()
    )
    ax = fig.add_subplot(111)
    sns.lineplot(data=dataf_daily, x="ts", y=metric, ax=ax)
    ax.set_title(f"{metric} по дням (Май 2026)")
    ax.set_xlabel("Дата")
    ax.set_ylabel(metric)
    fig.tight_layout()
    canvas.draw_idle()

def plot_bar():
    global cur_chart
    cur_chart = "bar"
    dataf = get_filtered()
    if dataf.empty:
        show_empty()

```

```

        return
    clear()
    metric = metric_var.get()
    agg     = agg_var.get()
    grouped = (
        dataf.groupby("acc_id")[metric].agg(agg).reset_index()
        .sort_values(metric, ascending=False).head(10)
    )
    ax = fig.add_subplot(111)
    sns.barplot(data=grouped, x="acc_id", y=metric, ax=ax)
    ax.set_title(f"Топ аккаунтов по {metric}")
    fig.tight_layout()
    canvas.draw_idle()

def plot_scatter():
    global cur_chart
    cur_chart = "scatter"
    dataf = get_filtered()
    if dataf.empty:
        show_empty()
        return
    clear()
    palette = {
        "Обычный": "gray", "Интересный": "orange", "Вирусный": "red"
    }
    ax = fig.add_subplot(111)
    sns.scatterplot(
        data=dataf, x="post_id", y="likes",
        hue="post_quality", palette=palette, alpha=0.6, ax=ax
    )
    ax.set_title("Зависимость лайков от ID поста")
    ax.set_xlabel("Глобальный ID поста")
    ax.set_ylabel("Количество лайков")
    fig.tight_layout()
    canvas.draw_idle()

```



```

def plot_heatmap():
    global cur_chart
    cur_chart = "heat"
    dataf = get_filtered()
    if dataf.empty():
        show_empty()
        return
    clear()
    corr = dataf[[
        "likes", "shares", "com", "engagement", "virality", "engagement_rate"
    ]].corr()
    ax = fig.add_subplot(111)
    sns.heatmap(corr, annot=True, cmap="coolwarm", ax=ax)
    ax.set_title("Корреляция признаков")
    fig.tight_layout()
    canvas.draw_idle()

def update_chart(event=None):
    if cur_chart == "line":
        plot_line()
    elif cur_chart == "bar":
        plot_bar()
    elif cur_chart == "scatter":
        plot_scatter()
    elif cur_chart == "heat":
        plot_heatmap()

```

Описание блока кода

Этот блок содержит ядро логики дашборда. Функция `get_filtered` возвращает срез датафрейма, отфильтрованный по текущим значениям из `acc_var` и `quality_var`. Вспомогательные функции `clear` и `show_empty` отвечают за очистку холста и отображение заглушки, если выборка пуста. Четыре основные функции `plot_line`, `plot_bar`, `plot_scatter`, `plot_heatmap` строят линейный график динамики метрик по дням, столбчатый график топ 10

аккаунтов, точечную диаграмму scatterplot с цветовой кодировкой по качеству постов и тепловую карту корреляции числовых признаков.

Диспетчер `update_chart` перерисовывает текущий активный график при изменении параметров фильтров.

Исходный код блока кода на Python

```
tk.Label(left, text="Аккаунт").pack(pady=5)
acc_list = ["Все"] + sorted(df_raw["acc_id"].unique())
acc_combo = ttk.Combobox(left, textvariable=acc_var, values=acc_list)
acc_combo.pack(pady=5)
acc_combo.bind("<<ComboboxSelected>>", update_chart)
```

```
tk.Label(left, text="Метрика").pack(pady=5)
metric_combo = ttk.Combobox(
    left, textvariable=metric_var,
    values=["engagement", "likes", "shares", "com"]
)
metric_combo.pack(pady=5)
metric_combo.bind("<<ComboboxSelected>>", update_chart)
```

```
tk.Label(left, text="Агрегация").pack(pady=5)
tk.Radiobutton(left, text="Sum", variable=agg_var, value="sum", command=update_c
tk.Radiobutton(left, text="Mean", variable=agg_var, value="mean", command=update_c
tk.Radiobutton(left, text="Median", variable=agg_var, value="median", command=update_c
```

```
tk.Label(left, text="Качество поста").pack(pady=5)
quality_combo = ttk.Combobox(
    left, textvariable=quality_var,
    values=["Все", "Обычный", "Интересный", "Вирусный"]
)
quality_combo.pack(pady=5)
quality_combo.bind("<<ComboboxSelected>>", update_chart)
```

```
tk.Button(left, text="Line", command=plot_line).pack(fill=tk.X, pady=5)
tk.Button(left, text="Bar", command=plot_bar).pack(fill=tk.X, pady=5)
```

```

tk.Button(left, text="Scatter", command=plot_scatter).pack(fill=tk.X, pady=5)
tk.Button(left, text="Heatmap", command=plot_heatmap).pack(fill=tk.X, pady=5)

def export_plot():
    path = filedialog.asksaveasfilename(defaultextension=".png")
    if path:
        fig.savefig(path, dpi=300, bbox_inches="tight")
        messagebox.showinfo("Экспорт", "Сохранено")

tk.Button(left, text="Экспорт", command=export_plot).pack(fill=tk.X, pady=20)

np.save("data_final33.npy", dataf_work)
print(dataf_work.columns.tolist())
print(dataf_work.dtypes)

plot_line()
root.mainloop()

```

Описание блока кода

На левой панели создаются выпадающие списки и кнопки для выбора аккаунта, метрики, типа агрегации и качества постов. К каждому элементу привязывается событие `update_chart` или прямая команда построения графика. Функция `export_plot`, которая открывает системный диалог для сохранения текущего графика в PNG

В самом конце обновленный датафрейм сохраняется на диск под именем `data_final33.npy`. При запуске программы вызывается стартовый линейный график, и запускается главный цикл обработки событий

Демонстрация работы

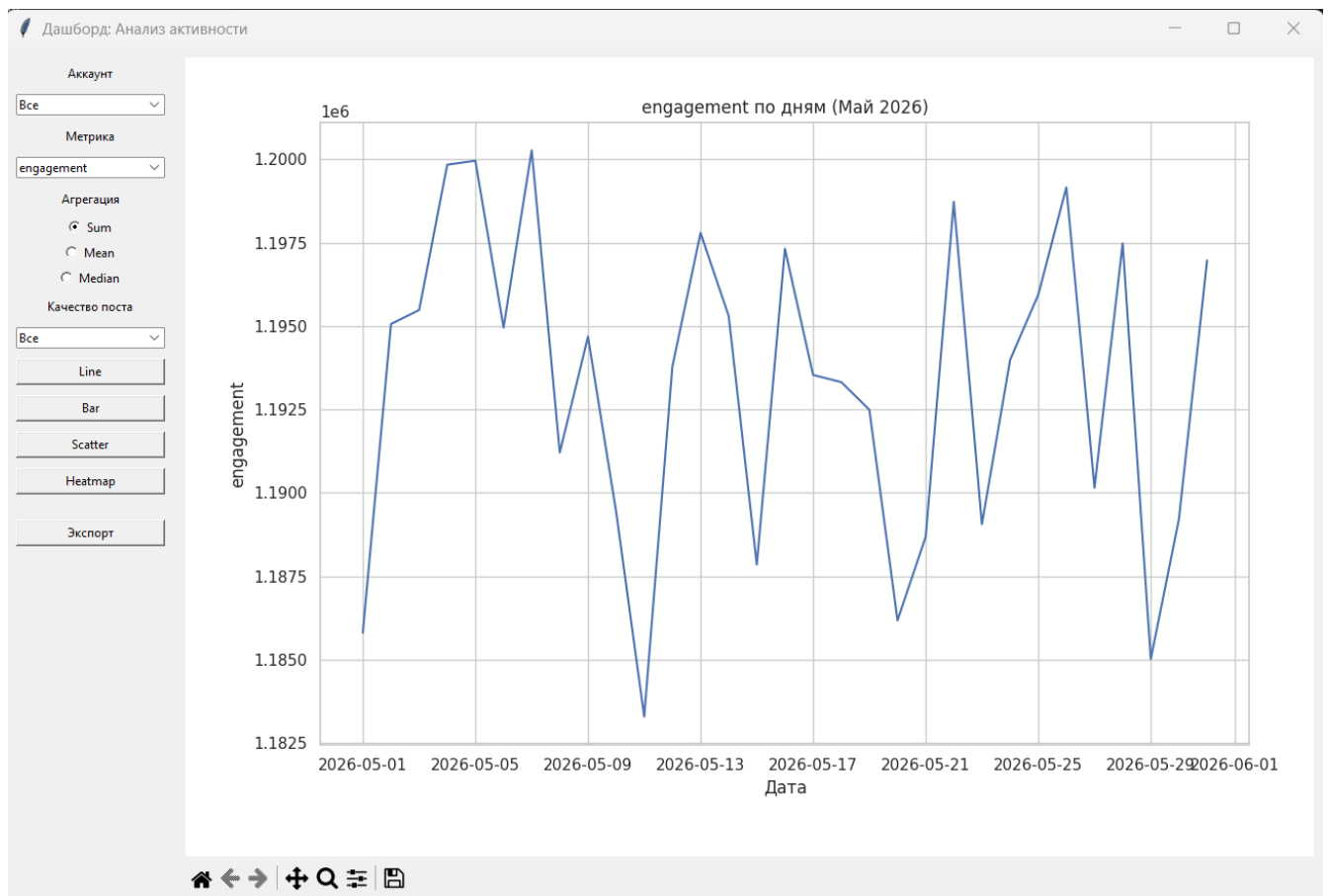


Рис. 1: Начальный экран, график метрики по дням

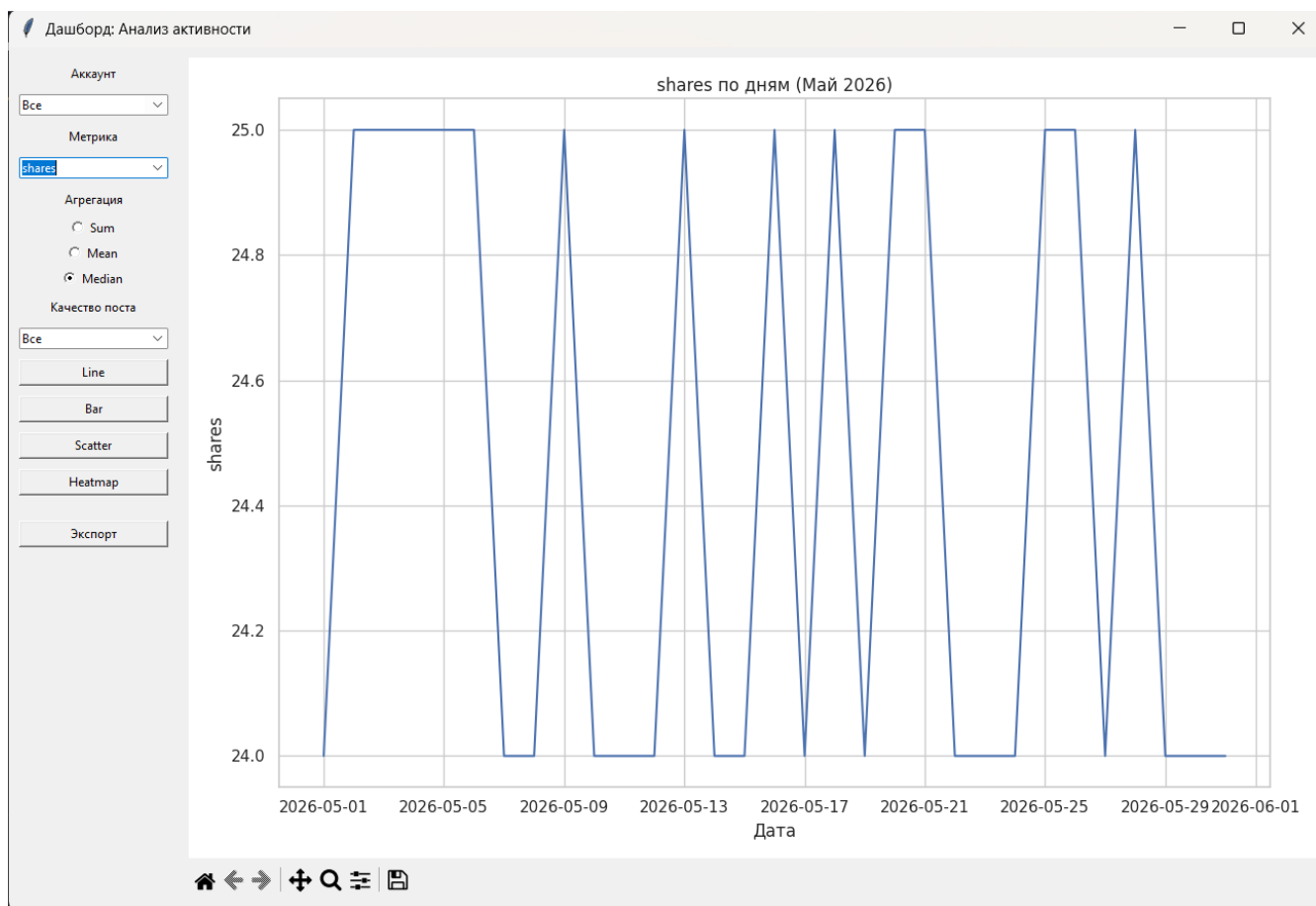


Рис. 2: Выбор по агрегации

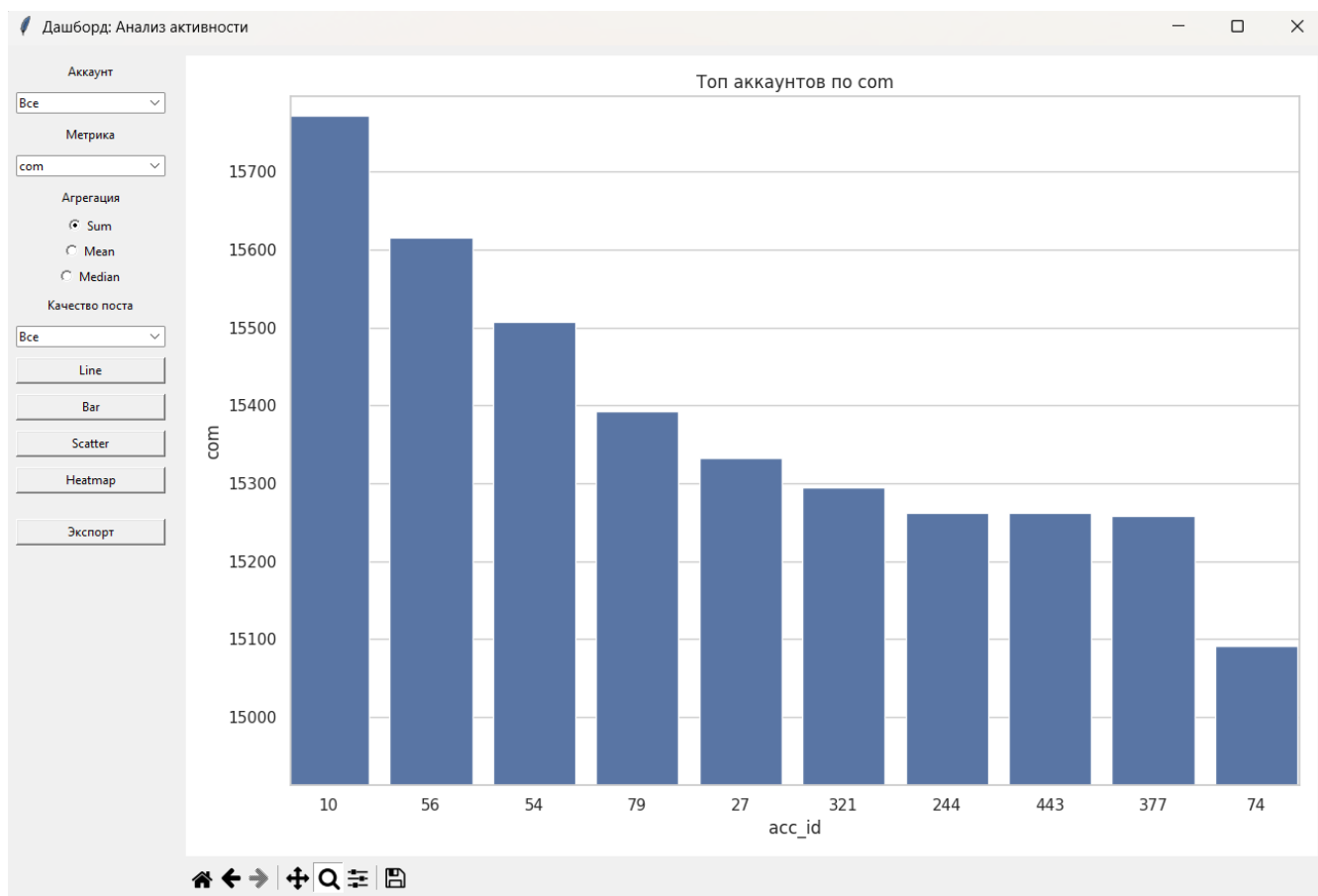


Рис. 3: График топ по метрике.

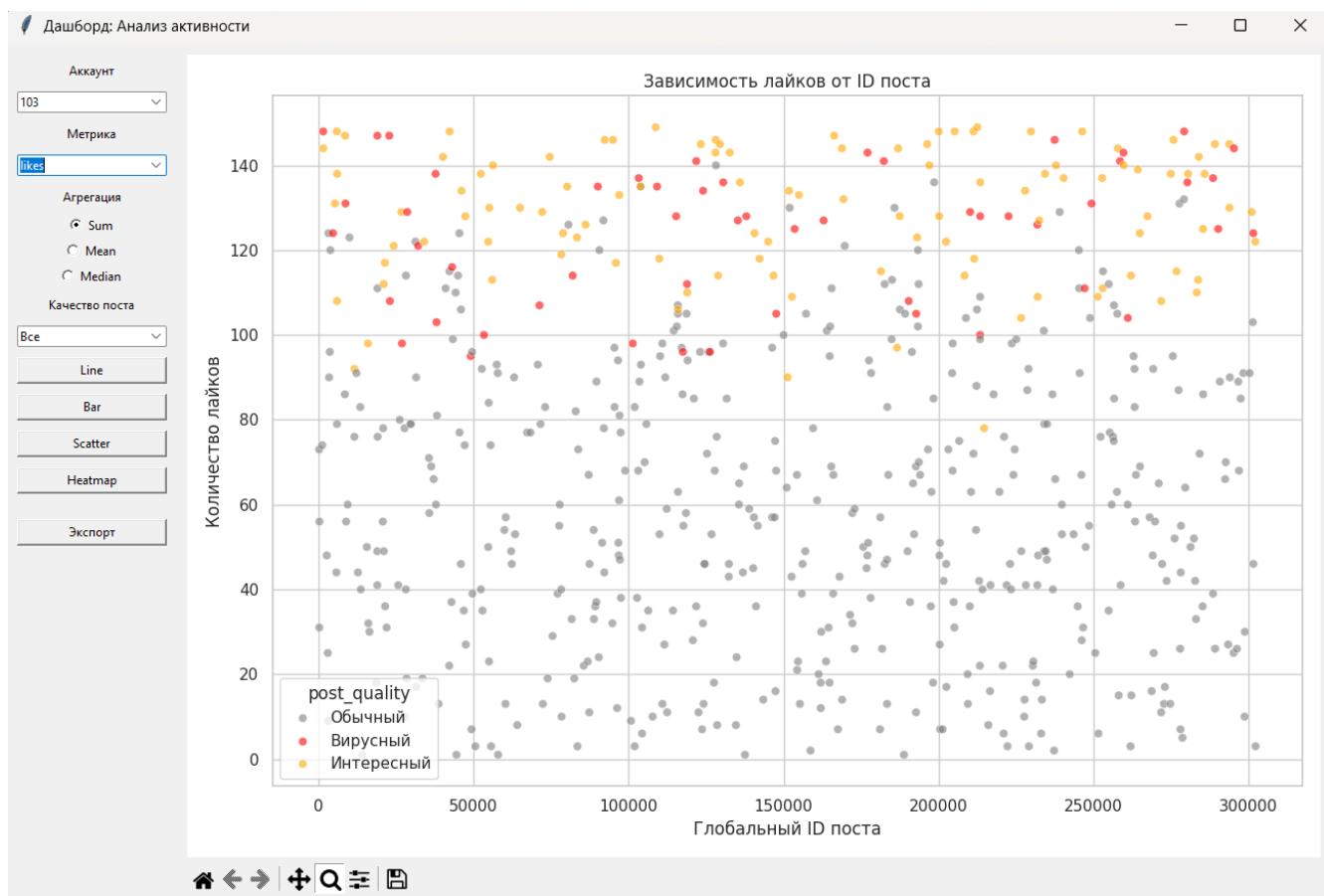


Рис. 4: Лайки на постах по ID.

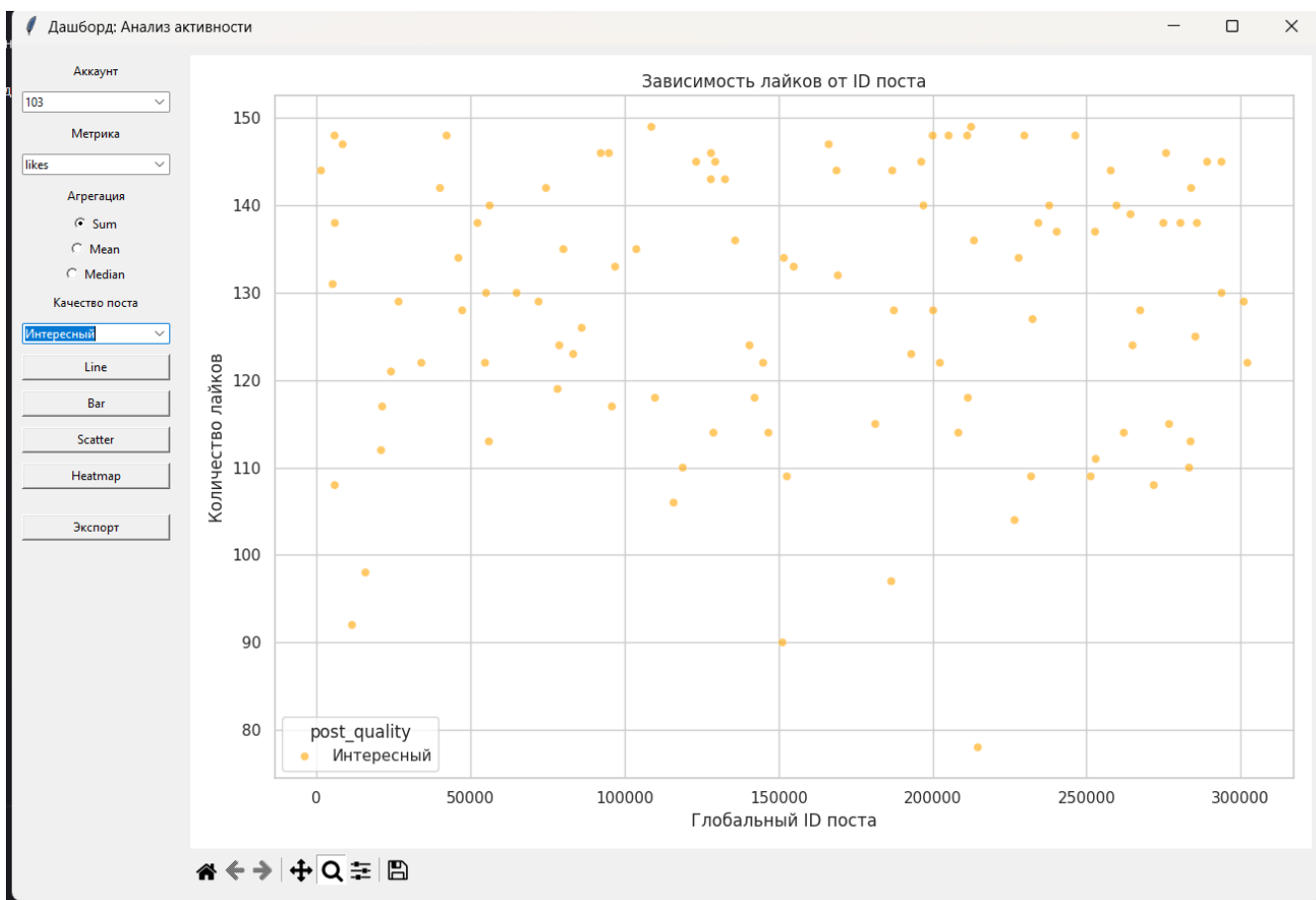


Рис. 5: Выбор качества поста.

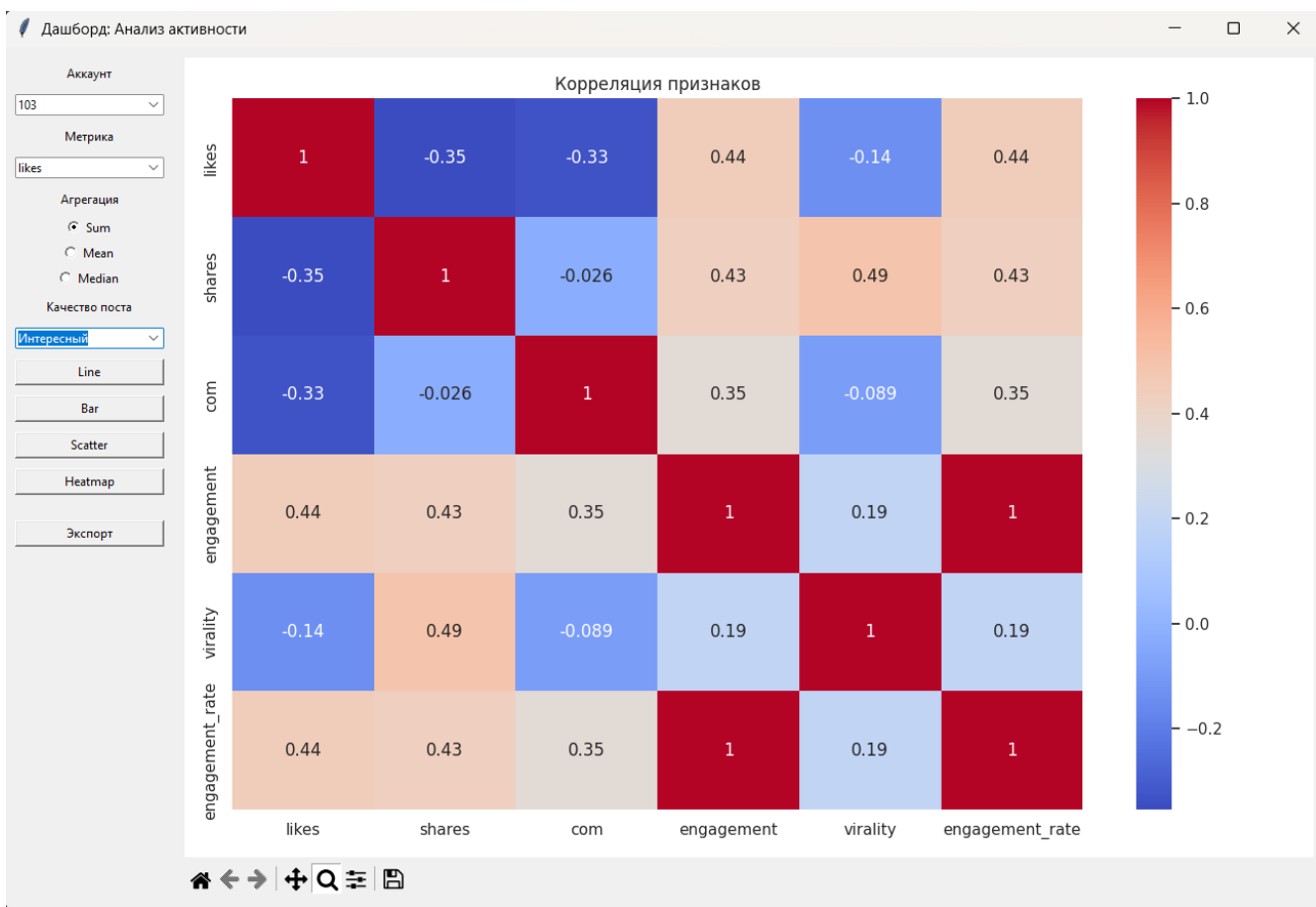


Рис. 6: Heatmap

Анализ результатов

Разработанный дашборд позволяет визуализировать данные. Тепловая карта показывает силу взаимосвязи между лайками, репостами и комментариями, а группировка по аккаунтам помогает определить авторов с наиболее хорошими метриками. Линейные графики демонстрируют суточные паттерны активности и общие тренды.

Поле тональности, заявленное в техническом задании, не интегрировано в интерфейс и графики. Интерфейс выполняет вычисления в главном потоке, что может приводит к зависаниям при обработке больших объемов данных.

Вывод

В ходе работы закрепились навыки обработки, очистки и добавления новых данных с помощью Pandas и NumPy. Освоено встраивание интерактивных графиков matplotlib и seaborn в десктопный gui на базе Tkinter, включая обработку событий, управление состоянием холста и обновление виджетов согласно с данными.

Основной сложностью стала корректная перерисовка графиков и наложений осей при переключении фильтров.

Необходимо добавить в дашборд анализ тональности (именно в качестве репутации) для оценки репутации аккаунтов и фильтрации негативного контента. Дополнительно стоит добавить возможность экспорта отфильтрованных данных и графиков в форматы csv, а также внедрить индикаторы загрузки, так как при перестроении вычисления происходят медленно.